

Lezione 15: Java Collections Framework, Equals/HashCode e Ordinamento

Questo notebook raccoglie tutto il codice della Lezione 15 (MavenDate):

- `Pair.java` , `OrderedPair.java` , `DatePair.java` , `DateInterval.java`
- `Date.java` (con implementazione coerente di `equals` , `hashCode` e `compareTo`)
- `DateInterval.java` (implementa `Comparable<DateInterval>`)
- `MainDate.java` (utilizzo di `List` , `ArrayList` , `Set` , `HashSet` , `Collections.sort` e `Comparator` personalizzati)

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/
├── codice-commentato/
│   └── Lezione15/
│       ├── MavenDate
│       │   ├── pom.xml
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   ├── it
│       │       │   │   │   ├── oop
│       │       │   │   │   │   ├── core
│       │       │   │   │   │   │   ├── AmericanDate.java
│       │       │   │   │   │   │   ├── BirthDay.java
│       │       │   │   │   │   │   ├── Date.java
│       │       │   │   │   │   │   ├── DateInterval.java
│       │       │   │   │   │   │   ├── DatePair.java
│       │       │   │   │   │   │   ├── FormattedDate.java
│       │       │   │   │   │   │   ├── FormattedDateConverter.java
│       │       │   │   │   │   │   ├── ItalianDate.java
│       │       │   │   │   │   │   ├── OrderedPair.java
│       │       │   │   │   │   │   ├── Pair.java
│       │       │   │   │   │   │   ├── Time.java
│       │       │   │   │   │   │   └── TimeStamp.java
│       │       │   │   │   │   └── ui
│       │       │   │   │   │       ├── Date.java
│       │       │   │   │   │       └── MainDate.java
│       │       │   └── resources
│       │       └── test
│       │           ├── java
│       │           │   ├── it
│       │           │   │   ├── oop
│       │           │   │   │   ├── core
│       │           │   │   │   │   └── TestItalianDate.java
```

Argomenti trattati:

- Il contratto `equals` e `hashCode` : perché è fondamentale sovrascriverli insieme quando gli oggetti vengono inseriti in strutture hash (`HashSet` , `HashMap`)
- Collezioni Java: `List<T>` (preserva duplicati e ordine di inserimento) vs `Set<T>` (nessun duplicato)
- Ordinamento naturale con `Comparable<T>` e ordinamento personalizzato con `Comparator<T>` (tramite espressioni Lambda)
- Ordinamento di intervalli (`DateInterval`) per data di inizio e data di fine

1. Interfaccia `Time`

```
In [1]: interface Time {
        int getSeconds();
        int getMinutes();
```

```
    int getHours();  
}
```

2. Classe Date con equals, hashCode e compareTo

```
In [2]: import java.util.Objects;  
  
class Date implements Comparable<Date> {  
    protected int day;  
    protected int month;  
    protected int year;  
  
    public Date(int day, int month, int year) {  
        this.day = day;  
        this.month = month;  
        this.year = year;  
        verify();  
    }  
    public Date(int day, int month) {  
        this(day, month, 2025);  
    }  
    public Date(Date other) {  
        this.day = other.day;  
        this.month = other.month;  
        this.year = other.year;  
        verify();  
    }  
  
    void verify() {  
        if (year < 0 || month < 1 || month > 12)  
            System.out.println("Illegal date!"); // Illegal date!  
        else  
            if (day < 1 || day > daysPerMonth(month))  
                System.out.println("Illegal date!"); // Illegal date!  
    }  
  
    public int getDay() { return day; }  
    public int getMonth() { return month; }  
    public int getYear() { return year; }  
  
    public static int daysPerMonth(int month) {  
        int days;  
        switch(month) {  
            case 4:  
            case 6:  
            case 9:  
            case 11:  
                days = 30;  
                break;  
            case 2:  
                days = 28;  
                break;  
            default:  
                days = 31;  
                break;  
        }  
        return days;  
    }  
  
    @Override  
    public String toString() {  
        return String.format("%dm%dd%d", year, month, day);  
    }  
  
    @Override  
    public boolean equals(Object other) {  
        if (other == null) return false;  
        if (this == other) return true;  
        if (!(other instanceof Date)) return false;  
        Date otherAsDate = (Date) other;  
        return this.day == otherAsDate.getDay() &&  
            this.month == otherAsDate.getMonth() &&  
            this.year == otherAsDate.getYear();  
    }  
}
```

```

    }

    @Override
    public int hashCode() {
        return Objects.hash(day, month, year);
    }

    @Override
    public int compareTo(Date otherAsDate) {
        int diff = this.year - otherAsDate.getYear();
        if (diff != 0) return diff;
        diff = this.month - otherAsDate.getMonth();
        if (diff != 0) return diff;
        return this.day - otherAsDate.getDay();
    }

    public static class Builder {
        private final int year;
        public Builder(int year) {
            this.year = (year > 0) ? year : 1970;
        }
        public Date build(int day, int month) {
            if (month < 1 || month > 12 || day < 1 || day > daysPerMonth(month))
                return new Date(1, 1, year);
            return new Date(day, month, year);
        }
    }
}

```

3. Classi `FormattedDate`, `FormattedDateConverter`, `ItalianDate`, `AmericanDate`, `TimeStamp` e `BirthDay`

```

In [3]: abstract class FormattedDate extends Date {
    protected final String format;
    protected final String[] months;

    public FormattedDate(int day, int month, int year, String format, String[] months) {
        super(day, month, year);
        this.format = format;
        this.months = months;
    }

    public final String printFormat() { return format; }
    public final String getMonthAsString() { return months[getMonth()-1]; }
    public abstract String prettyPrint();
}

@FunctionalInterface
interface FormattedDateConverter {
    FormattedDate convert(FormattedDate date);
}

class ItalianDate extends FormattedDate {
    private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
    "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };
    public ItalianDate(int day, int month, int year) {
        super(day, month, year, "dd/mm/yyyy", MONTHS_IT);
    }
    @Override
    public String prettyPrint() { return day + " " + getMonthAsString() + " " + getYear(); }
    @Override
    public String toString() { return day + "/" + getMonth() + "/" + getYear(); }
}

class AmericanDate extends FormattedDate {
    private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December" };
    public AmericanDate(int day, int month, int year) {
        super(day, month, year, "mm/dd/yyyy", MONTHS_US);
    }
    @Override
    public String prettyPrint() { return getMonthAsString() + " " + day + ", " + getYear(); }
}

```

```

    @Override
    public String toString() { return getMonth() + "/" + getDay() + "/" + getYear(); }
}

class TimeStamp extends Date implements Time {
    protected final int seconds, minutes, hours;
    public TimeStamp(int seconds, int minutes, int hours, int day, int month, int year) {
        super(day, month, year);
        this.seconds = seconds; this.minutes = minutes; this.hours = hours;
    }
    @Override public int getSeconds() { return seconds; }
    @Override public int getMinutes() { return minutes; }
    @Override public int getHours() { return hours; }
    @Override public String toString() {
        return String.format("%s[%02d:%02d:%02d]", super.toString(), hours, minutes, seconds);
    }
}

class Birthday extends Date {
    private static Birthday instance = null;
    private Birthday() { super(1, 1, 1970); }
    public static Birthday getInstance() {
        if (instance == null) instance = new Birthday();
        return instance;
    }
    public String prettyPrint() { return "1 gennaio 1970"; }
}

```

4. Classi Generiche e `DateInterval` con `Comparable`

In [4]:

```

class Pair<F, S> {
    private final F first;
    private final S second;
    public Pair(F first, S second) { this.first = first; this.second = second; }
    public F getFirst() { return first; }
    public S getSecond() { return second; }
}

class OrderedPair<T extends Comparable<T>> {
    private final T first;
    private final T second;
    public OrderedPair(T first, T second) {
        this.first = first;
        this.second = second;
        if (first.compareTo(second) > 0)
            System.out.println("Pair not ordered!"); // Pair not ordered!
    }
    public T getFirst() { return first; }
    public T getSecond() { return second; }
}

class DatePair extends Pair<Date, Date> {
    public DatePair(Date left, Date right) { super(left, right); }
    public Date getLeft() { return getFirst(); }
    public Date getRight() { return getSecond(); }
    @Override public String toString() {
        return String.format("[%s .. %s]", getLeft().toString(), getRight().toString());
    }
}

class DateInterval extends OrderedPair<Date> implements Comparable<DateInterval> {
    public DateInterval(Date left, Date right) { super(left, right); }
    public Date getLeft() { return getFirst(); }
    public Date getRight() { return getSecond(); }
    @Override public String toString() {
        return String.format("[%s .. %s]", getLeft().toString(), getRight().toString());
    }
    @Override public int compareTo(DateInterval other) {
        int diff = this.getLeft().compareTo(other.getLeft());
        if (diff != 0) return diff;
        return this.getRight().compareTo(other.getRight());
    }
}

```

```
}  
}
```

5. Classe MainDate ed Esecuzione

```
In [5]: import java.util.*;  
  
class MainDate {  
    public static void main(String[] args) {  
        Date.Builder dateBuilder = new Date.Builder(2025);  
        Date d1 = dateBuilder.build(10, 9);  
        Date d2 = dateBuilder.build(10, -1);  
        System.out.println(d1.toString()); // y2025m9d10  
        System.out.println(d2.toString()); // y2025m1d1  
  
        Time init = new Time() {  
            @Override public int getHours() { return 0; }  
            @Override public int getMinutes() { return 0; }  
            @Override public int getSeconds() { return 0; }  
            @Override public String toString() {  
                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());  
            }  
        };  
        System.out.println(init.toString()); // 00:00:00  
  
        FormattedDateConverter toAmerican =  
            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());  
        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof AmericanDate); //  
true  
  
        Pair<String, FormattedDate> event = new Pair<>("OOP exam", new ItalianDate(2, 2, 2026));  
        System.out.println(event.getFirst() + " on " + event.getSecond().prettyPrint()); // OOP exam on 2  
febbraio 2026  
  
        DatePair dp = new DatePair(new Date(1, 2, 2025), new Date(1, 1, 2025));  
        System.out.println("date pair: " + dp.toString()); // date pair: [y2025m2d1 .. y2025m1d1]  
  
        DateInterval di = new DateInterval(new Date(1, 2, 2025), new Date(1, 1, 2025)); // costruttore  
OrderedPair stampa: Pair not ordered!  
  
        Pair unknown = new Pair("OOP exam", new ItalianDate(2, 2, 2026));  
        System.out.println(unknown.getFirst().toString() + " on " + ((ItalianDate)  
unknown.getSecond()).prettyPrint()); // OOP exam on 2 febbraio 2026  
  
        // Collezioni: List vs Set  
        List<Date> dateList = new ArrayList<>();  
        dateList.add(new AmericanDate(7, 11, 2025));  
        dateList.add(new Date(9, 11, 2025));  
        dateList.add(new Timestamp(0, 0, 0, 8, 11, 2025));  
        dateList.add(new AmericanDate(7, 11, 2025)); // duplicato ammesso in List  
        System.out.println("list(" + dateList.size() + "): " + dateList.toString()); // list(4):  
[11/7/2025, y2025m11d9, y2025m11d8[00:00:00], 11/7/2025]  
  
        Set<Date> dateSet = new HashSet<>(dateList); // Set elimina duplicati grazie a equals e hashCode  
        System.out.println("set(" + dateSet.size() + "): " + dateSet.toString()); // set(3): [11/7/2025,  
y2025m11d9, y2025m11d8[00:00:00]]  
  
        Date usD1 = new AmericanDate(1, 1, 1970);  
        Date usD2 = new AmericanDate(1, 1, 1970);  
        System.out.println("usD1 != usD2: " + usD1.equals(usD2)); // usD1 != usD2: true  
        System.out.println("hash(usD1): " + usD1.hashCode()); // hash(usD1): 36450  
        System.out.println("hash(usD2): " + usD2.hashCode()); // hash(usD2): 36450  
  
        BirthDay bDay = BirthDay.getInstance();  
        System.out.println(bDay.prettyPrint()); // 1 gennaio 1970  
  
        Comparator<Date> c = (da, db) -> da.compareTo(db);  
        Comparator<Date> c2 = Date::compareTo;  
        System.out.println(c.toString()); // it.oop.ui.MainDate$$Lambda/0x000000009237bb48@7bd325a4  
        System.out.println(c2.toString()); // it.oop.ui.MainDate$$Lambda/0x000000009237b920@677a5bd  
  
        // Ordinamento naturale su List<Date>  
        Collections.sort(dateList);
```

```

        System.out.println(dateList.toString()); // [11/7/2025, 11/7/2025, y2025m11d8[00:00:00],
y2025m11d9]

        // Ordinamento di intervalli
        List<DateInterval> intvList = new ArrayList<>();
        intvList.add(new DateInterval(new Date(1, 1, 2025), new Date(20, 1, 2025)));
        intvList.add(new DateInterval(new Date(10, 1, 2025), new Date(31, 1, 2025)));
        System.out.println(intvList.toString()); // [[y2025m1d1 .. y2025m1d20], [y2025m1d10 ..
y2025m1d31]]
        Collections.sort(intvList);
        System.out.println(intvList.toString()); // [[y2025m1d10 .. y2025m1d31], [y2025m1d1 ..
y2025m1d20]]
    }
}
MainDate.main(null);

```

```

y2025m9d10
y2025m1d1
00:00:00
true
OOP exam on 2 febbraio 2026
date pair: [y2025m2d1 .. y2025m1d1]
Pair not ordered!
OOP exam on 2 febbraio 2026
list(4): [11/7/2025, y2025m11d9, y2025m11d8[00:00:00], 11/7/2025]
set(3): [11/7/2025, y2025m11d8[00:00:00], y2025m11d9]
usD1 ?= usD2: true
hash(usD1): 32753
hash(usD2): 32753
1 gennaio 1970
REPL.$JShell$15$MainDate$$Lambda/0x000000001924e688@53ffca79
REPL.$JShell$15$MainDate$$Lambda/0x000000001924e928@195069f2
[11/7/2025, 11/7/2025, y2025m11d8[00:00:00], y2025m11d9]
[[y2025m1d1 .. y2025m1d20], [y2025m1d10 .. y2025m1d31]]
[[y2025m1d1 .. y2025m1d20], [y2025m1d10 .. y2025m1d31]]

```